

AfriMarkets

Guide d'architecture & d'extension

Comment utiliser la structure du package et configurer de nouveaux marchés avec toutes les fonctions associées.

Auteurs	Koffi Frederic SESSIE — Olabiyi Aurel Geoffroy ODJO
Version	1.0
Date	16/05/2026
Package	afrimarkets (Python)

Sommaire

1. Vue d'ensemble de l'architecture
2. Structure du package
3. La classe **Market** — attributs de base
4. Le **MarketRegistry** — registre central
5. Configuration manuelle d'un marché
6. Ajouter un nouveau marché (étape par étape)
7. Fonctions associées : **get_tickers**, **get_data**
8. Pattern **MarketAdapter** (avancé)
9. Tests et validation
10. Checklist d'implémentation
11. Commandes utiles
12. FAQ

1. Vue d'ensemble de l'architecture

Le package **AfriMarkets** fournit une API unifiée pour interagir avec les principales bourses africaines (BRVM, NGX, JSE, EGX, etc.). Il repose sur trois briques fondamentales conçues pour rester simples à étendre.

- **Market** — une dataclass décrivant les attributs de base d'un marché (identité, URLs, DataFrames d'actifs).
- **MarketRegistry** — un registre central qui mémorise tous les marchés configurés et permet d'y accéder par leur code court (ex. `registry["BRVM"]`).
- **MarketAdapter** (optionnel, pattern avancé) — une classe abstraite qui définit l'interface des fonctions de récupération de données par marché.

L'enregistrement des marchés est **manuel** : chaque marché est instancié explicitement, puis ajouté au registre via `registry.register(...)` ou via la fonction d'initialisation `create_all_markets()`.

2. Structure du package

Voici l'arborescence recommandée pour le package, qui sépare clairement le cœur (classes de base) des implémentations spécifiques à chaque marché.

```
AfriMarkets/
├── afrimarkets/
│   ├── __init__.py          # API publique
│   ├── market.py            # Classe Market (dataclass)
│   ├── registry.py          # MarketRegistry + create_all_markets()
│   └── markets/              # (optionnel) implémentations par marché
│       ├── base.py          # classe abstraite MarketAdapter
│       └── brvm/
│           ├── adapter.py    # BRVMAdapter
│           └── legacy/scraper.py
│               └── ngx/...
│   └── utils/                # helpers communs (dates, HTTP, cache)
├── examples/
├── demo_markets.py
├── tests/
│   └── test_registry.py
├── docs/
├── pyproject.toml
├── setup.py
└── requirements.txt
```

3. La classe Market — attributs de base

La classe **Market** (dans `afrimarkets/market.py`) est une *dataclass* qui décrit n'importe quel marché du package. Elle est l'équivalent Python de la classe `S4 african_market` du package R original.

Attributs

Attribut	Type	Description
<code>short_name</code>	<code>str</code>	Code court (ex. BRVM, NGX). Normalisé en MAJUSCULES.

full_name	str	Nom officiel complet du marché.
official_url	str	URL principale du site officiel.
market_url	str	Chemin ou URL spécifique de la section marché.
market_data_url	list[str]	Liste d'URLs pour récupérer les données.
list_url	str	URL générale de listing.
list_shares_url	str	URL de listing des actions.
list_indexes_url	str	URL de listing des indices.
list_bonds_url	str	URL de listing des obligations.
indexes	pd.DataFrame	DataFrame des indices disponibles.
shares	pd.DataFrame	DataFrame des actions disponibles.
bonds	pd.DataFrame	DataFrame des obligations disponibles.
ticker_full_name	str	Convention de nommage des tickers.

Méthodes utilitaires

- `has_indexes()` / `has_shares()` / `has_bonds()` — booléens indiquant si les DataFrames sont peuplés.
- `summary()` — renvoie un dict léger avec le nom, l'URL et les comptages d'actifs (utile pour le logging).
- `__repr__` et `__str__` — affichage lisible dans la console et les logs.

Exemple d'instanciation

```
from afrimarkets import Market

brvm = Market(
    short_name="BRVM",
    full_name="BOURSE REGIONALE DES VALEURS MOBILIERES",
    official_url="https://www.brvm.org",
    market_url="/equities/ivory-coast",
    market_data_url=[
        "https://www.sikafinance.com/api/general/GetHistos",
        "/equities/ivory-coast",
    ],
)
print(brvm)
# BRVM - BOURSE REGIONALE DES VALEURS MOBILIERES (https://www.brvm.org)
```

4. Le MarketRegistry — registre central

Le **MarketRegistry** est un conteneur qui stocke tous les marchés configurés dans le package. Une instance globale, `registry`, est créée à l'import de `afrimarkets` et automatiquement peuplée par `create_all_markets()`.

API du registre

Méthode	Description
<code>register(market)</code>	Ajoute un Market au registre (KO si déjà présent).
<code>unregister(name)</code>	Retire un marché par son code court.
<code>get(name)</code>	Renvoie un Market ou None.
<code>all()</code>	Retourne la liste de tous les marchés.
<code>names()</code>	Retourne la liste des codes courts enregistrés.
<code>clear()</code>	Vide le registre.
<code>registry[name]</code>	Accès direct par clé (lève <code>KeyError</code> si inconnu).
<code>name in registry</code>	Test d'appartenance.
<code>for m in registry</code>	Itération sur les marchés.
<code>len(registry)</code>	Nombre de marchés enregistrés.

Exemples d'utilisation

```
from afrimarkets import registry

# Accès par code court
brvm = registry["BRVM"]
print(brvm.official_url)           # https://www.brvm.org

# Itération
for market in registry:
    print(market.short_name, "->", market.full_name)

# Test d'appartenance
"NGX" in registry                  # True
len(registry)                     # 10 marchés par défaut

# Récupérer la liste des codes
registry.names()
# ['GSE', 'NGX', 'JSE', 'MSE', 'EGX', 'NSE', 'BRVM', 'TSE', 'BSE', 'SEM']
```

5. Configuration manuelle d'un marché

Conformément au choix d'architecture, chaque marché est configuré **manuellement** dans `registry.py`, à l'intérieur de la fonction `_default_markets()`. Cette approche est explicite, traçable, et facile à relire.

```
# afrimarkets/registry.py

def _default_markets() -> list[Market]:
    return [
        Market(
            short_name="BRVM",
            full_name="BOURSE REGIONALE DES VALEURS MOBILIERES",
```

```
        official_url="https://www.brvm.org",
        market_url="/equities/ivory-coast",
        market_data_url=[
            "https://www.sikafinance.com/api/general/GetHistos",
            "/equities/ivory-coast",
        ],
    ),
    # ... un Market(...) par marché ...
]
```

Au démarrage du package, `create_all_markets()` boucle sur cette liste et appelle `registry.register(m)` pour chaque marché. La fonction est **idempotente** : on peut la rappeler sans doublons.

6. Ajouter un nouveau marché

Voici la procédure complète, étape par étape, pour intégrer un nouveau marché (exemple fictif **XYZ**).

Étape 1 — Ajouter la configuration dans le registre

```
# afrimarkets/registry.py

def _default_markets() -> list[Market]:
    return [
        # ... marchés existants ...
        Market(
            short_name="XYZ",
            full_name="XYZ STOCK EXCHANGE",
            official_url="https://www.xyz.example.com",
            market_url="/equities/xyz-country",
            market_data_url=["https://api.xyz.example.com/v1/data"],
            list_shares_url="/equities/xyz-country/shares",
            list_indexes_url="/equities/xyz-country/indexes",
            list_bonds_url="/equities/xyz-country/bonds",
            ticker_full_name="XYZ:{ticker}",
        ),
    ]
```

Étape 2 — Vérifier l'enregistrement

```
from afrimarkets import registry

assert "XYZ" in registry
print(registry["XYZ"].summary())
# {'short_name': 'XYZ', 'full_name': 'XYZ STOCK EXCHANGE', ...}
```

Étape 3 — Peupler les DataFrames (optionnel)

À la création, les attributs `shares`, `indexes` et `bonds` sont des DataFrames vides. Tu peux les remplir manuellement, depuis un fichier CSV, une API ou un scraper.

```
import pandas as pd
from afrimarkets import registry

xyz = registry["XYZ"]

# Depuis un CSV local
xyz.shares = pd.read_csv("data/xyz_shares.csv")
xyz.indexes = pd.read_csv("data/xyz_indexes.csv")

# Ou depuis une API
import requests
resp = requests.get("https://api.xyz.example.com/v1/shares")
xyz.shares = pd.DataFrame(resp.json())

print(xyz.has_shares()) # True
```

7. Fonctions associées : `get_tickers`, `get_data`

Pour exposer une API uniforme, le package fournit deux fonctions de haut niveau : `get_tickers` et `get_data`. Elles s'appuient sur le registre.

Squelette des fonctions

```
# afrimarkets/api.py
import pandas as pd
from typing import Optional
from .registry import registry
from .market import Market

def get_tickers(market_code: str) -> Market:
    """Retourne le Market peuplé avec ses tickers (shares/indexes/bonds)."""
    market = registry.get(market_code)
    if market is None:
        raise KeyError(f"Marché inconnu : {market_code!r}")

    # Stratégie de récupération propre au marché.
    # Ici, on délègue à un adaptateur si disponible, sinon on renvoie tel quel.
    adapter = _get_adapter(market_code)
    if adapter is not None:
        adapter.populate(market)
    return market

def get_data(
    market_code: str,
    ticker: str,
    from_date: Optional[str] = None,
    to_date: Optional[str] = None,
    period: str = "daily",
) -> pd.DataFrame:
    """Retourne un DataFrame OHLCV pour un ticker."""
    adapter = _get_adapter(market_code)
    if adapter is None:
        raise NotImplementedError(
            f"Aucun adaptateur de données pour {market_code}."
        )
    return adapter.fetch_ohlcvticker, from_date, to_date, period)
```

Le DataFrame renvoyé par `get_data` respecte le schéma standard : colonnes **Date**, **Ticker**, **Open**, **High**, **Low**, **Close**, **Volume**. Cette convention permet de chaîner les marchés sans transformation côté utilisateur.

8. Pattern MarketAdapter (avancé)

Pour les marchés qui ont besoin d'une logique de récupération spécifique (scraping, API REST, parsing), on définit une classe abstraite **MarketAdapter** dans `afrimarkets/markets/base.py`.

```
# afrimarkets/markets/base.py
from abc import ABC, abstractmethod
from datetime import date, datetime
from typing import Optional, Tuple
import pandas as pd
from ..market import Market

class MarketAdapter(ABC):
    """Interface commune pour tous les adaptateurs de marché."""

    market_code: str # à surcharger dans chaque sous-classe

    @abstractmethod
    def populate(self, market: Market) -> None:
        """Remplit market.shares / market.indexes / market.bonds."""

    @abstractmethod
    def fetch_ohlcv(
        self,
        ticker: str,
        from_date: Optional[str],
        to_date: Optional[str],
        period: str,
    ) -> pd.DataFrame:
        """Retourne OHLCV au schéma standard."""

    # ----- helpers communs -----
    @staticmethod
    def normalize_dates(
        from_date: Optional[str], to_date: Optional[str]
    ) -> Tuple[date, date]:
        today = date.today()
        fd = datetime.fromisoformat(from_date).date() if from_date else today
        td = datetime.fromisoformat(to_date).date() if to_date else today
        return fd, td
```

Implémentation pour BRVM

```
# afrimarkets/markets/brvm/adapter.py
from ..base import MarketAdapter
from ..market import Market
from .legacy.scrapier import fetch_shares, fetch_ohlcv

class BRVMAdapter(MarketAdapter):
    market_code = "BRVM"

    def populate(self, market: Market) -> None:
        market.shares = fetch_shares()

    def fetch_ohlcv(self, ticker, from_date, to_date, period):
        fd, td = self.normalize_dates(from_date, to_date)
        return fetch_ohlcv(ticker, fd, td, period)
```

Brancher l'adaptateur sur le registre

```
# afrimarkets/markets/__init__.py
from .brvm.adapter import BRVMAdapter
```

```
# ... importer les autres adaptateurs ...

ADAPTERS = {
    "BRVM": BRVMAdapter(),
    # "NGX": NGXAdapter(),
    # ...
}

def _get_adapter(market_code: str):
    return ADAPTERS.get(market_code.upper())
```

9. Tests et validation

Chaque marché doit avoir un test minimal qui vérifie son enregistrement et le bon fonctionnement de `get_tickers` / `get_data`.

```
# tests/test_xyz.py
import pytest
from afrimarkets import registry, get_tickers, get_data

def test_xyz_registered():
    assert "XYZ" in registry

def test_xyz_metadata():
    m = registry["XYZ"]
    assert m.full_name == "XYZ STOCK EXCHANGE"
    assert m.official_url.startswith("https://")

def test_xyz_get_tickers():
    m = get_tickers("XYZ")
    assert m.has_shares()

def test_xyz_get_data():
    df = get_data("XYZ", ticker="TICK1", from_date="2026-01-01")
    assert {"Date", "Close", "Volume"}.issubset(df.columns)
    assert not df.empty
```

10. Checklist d'implémentation

- 1. Ajouter le Market(...) dans `_default_markets()` de `registry.py`.
- 2. Vérifier que `"XYZ" in registry` renvoie `True`.
- 3. Renseigner les attributs URLs (`official_url`, `market_data_url`, etc.).
- 4. Si besoin de récupération dynamique, créer `markets/xyz/adapter.py`.
- 5. Implémenter `populate()` et `fetch_ohlcv()`.
- 6. Brancher l'adaptateur dans `markets/__init__.py`.
- 7. Créer le scraper / client API dans `markets/xyz/legacy/`.
- 8. Écrire les tests dans `tests/test_xyz.py`.
- 9. Lancer `pytest tests/test_xyz.py -v`.
- 10. Documenter les sources de données dans le README ou docs/.

11. Commandes utiles

```
# Lister les marchés enregistrés
python -c "from afrimarkets import registry; print(registry.names())"

# Tester rapidement un marché
python -c "from afrimarkets import registry; print(registry['BRVM'])"

# Exécuter tous les tests
pytest tests/ -v

# Tests d'un marché spécifique
pytest tests/test_brvm.py -v

# Valider la configuration globale
python validate_config.py

# Afficher la structure du projet
python STRUCTURE.py
```

12. FAQ

Q. Comment ajouter un marché sans scraper ?

R. Crée simplement le `Market(...)` dans `_default_markets()`. Tu peux peupler `shares/indexes/bonds` depuis un CSV au moment de l'utilisation.

Q. Comment gérer l'authentification d'une API ?

R. Mets les clés dans des variables d'environnement et lis-les avec `os.getenv("XYZ_API_KEY")` dans le scraper. Ne committe jamais de secrets.

Q. Que faire si l'API échoue (timeouts, erreurs réseau) ?

R. Implémente un `retry` avec `backoff` exponentiel (ex. `tenacity`) dans la couche `scraper`, et lève une exception métier claire au-dessus.

Q. Comment ajouter du cache ?

R. `functools.lru_cache` pour les helpers purs, ou un cache disque (`joblib`, `diskcache`) pour les requêtes HTTP coûteuses. Centralise dans `utils/cache.py`.

Q. Comment étendre l'interface ?

R. Ajoute des méthodes à `MarketAdapter` (ex. `get_live_quotes()`, `get_company_info()`) puis expose-les via `api.py`.

Q. Comment gérer plusieurs devises ou fuseaux ?

R. Ajoute des attributs optionnels à `Market` (`currency`, `timezone`) et applique la conversion dans le scraper avant de retourner le `DataFrame`.